

PHASE 2 OF 8

Prerequisites, Setup & Architecture

What you need installed, how to launch both services, and how a request flows through the system.

- Prerequisites
- Setup
- Workflow
- Structure

3. Prerequisites

Requirement	Why it matters
Python and pip installed	Runs both the FastAPI backend and the Streamlit frontend.
Basic Python and REST API knowledge	Helps you read the routes, schemas and request pattern.
Dependencies from requirements.txt	Installs FastAPI, Streamlit, google-genai, python-docx, ReportLab and python-dotenv.
Gemini API credential in .env	Lets the backend call Gemini without hard-coding the key.
Modern web browser	Streamlit serves the interface at localhost:8501.

3.1 Environment Setup Commands

The commands below use Windows PowerShell paths, as used in the project demonstration.

Create and activate a virtual environment

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Install project dependencies

```
pip install -r requirements.txt
```

Run the FastAPI backend

```
.\.venv\Scripts\python.exe -m uvicorn backend.main:app --reload
```

Run the Streamlit frontend (second terminal)

```
.\.venv\Scripts\python.exe -m streamlit run frontend\app.py
```

Keep the key private

The actual API key is never printed in this documentation. The local .env file must stay private and out of version control.

4. Project Workflow

Every document follows the same seven-step journey from form to export.

Step	What happens	Component
1	User opens Streamlit.	Browser
2	User enters document details.	Streamlit form
3	Streamlit sends a POST request to /generate.	Streamlit → FastAPI
4	FastAPI validates the request.	Pydantic schema
5	Gemini generates the draft.	Gemini generator
6	The result is displayed and made editable.	Streamlit
7	TXT, DOCX and PDF exports are prepared.	Export services

4.1 Architecture

USER → STREAMLIT → FASTAPI /generate → GEMINI → DOCUMENT → EDIT → TXT / DOCX / PDF

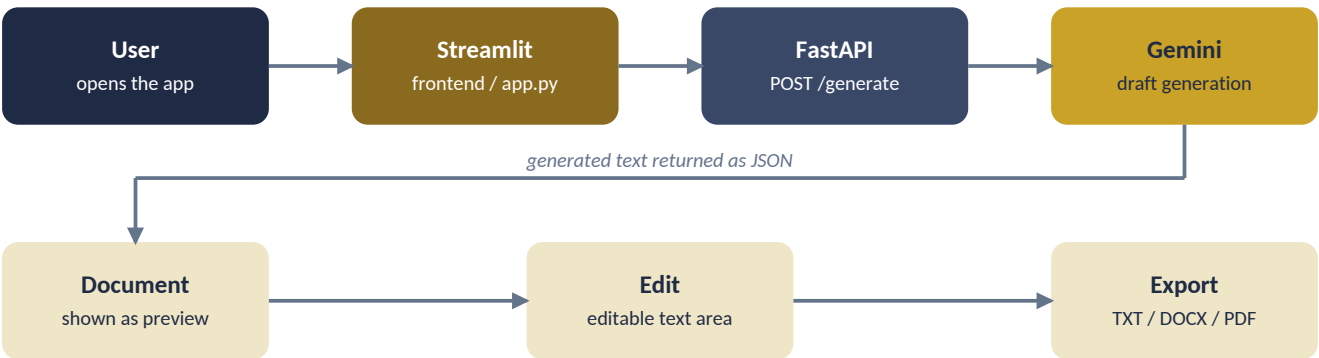


Figure — LegalEase request flow: from user input to exported document.

Layer	Responsibility
Frontend (Streamlit)	Collects input, shows the generated draft, hosts the editable text area and download buttons.
Backend (FastAPI)	Validates requests, exposes /generate, reports errors, enables CORS for the frontend.
AI core (Gemini)	Builds the prompt and calls the configured Gemini model.
Export services	Turn the edited text into TXT, DOCX and PDF files.

4.2 Project Structure

```
backend/
+-- main.py           # FastAPI app setup, CORS, status endpoints
+-- routes.py         # /generate endpoint
+-- schemas.py        # Pydantic request model
+-- config.py         # configuration loading
+-- ai_core/
|   +-- gemini_generator.py # Gemini integration
+-- services/
|   +-- document_service.py # document service logic
|   +-- exporters.py       # TXT / DOCX / PDF export utilities
+-- frontend/
+-- app.py            # Streamlit user interface
+-- tests/            # API and exporter tests
+-- .env.example      # safe configuration template
+-- requirements.txt   # Python dependencies
```

4.3 File Responsibilities

File	Responsibility
backend/main.py	Initializes FastAPI, enables local CORS, includes the document routes, exposes status endpoints.
backend/routes.py	Defines the /generate API route.
backend/schemas.py	Pydantic request validation for document details.
backend/config.py	Loads configuration such as the API key and model name.
backend/ai_core/gemini_generator.py	Gemini integration: prompt construction and model call.
backend/services/document_service.py	Document service logic.
backend/services/exporters.py	TXT, DOCX and PDF export utilities.
frontend/app.py	Streamlit interface: form, preview, editing and downloads.
tests/	API and exporter tests.

Reading order for new contributors
Start with frontend/app.py to see what the user does, then routes.py and schemas.py for the API contract, then gemini_generator.py and exporters.py.